



EXPLORING CAR FEATURES AND MSRP: A COMPREHENSIVE EDA IN PYTHON



TABLE OF CONTENT

- INTRODUCTION
- OBJECTIVE
- DATA OVERVIEW
- DATA CLEANING AND PREPARATION
- DATA ANALYSIS & VISUALIZATION
- ADVANCED BINNING AND GROUPING
- OUTLIER DETECTION AND HANDLING





INTRODUCTION

Exploratory Data Analysis (EDA) involves understanding datasets by summarizing their key characteristics, often through visualizations. This crucial step helps in uncovering insights and defining the problem statement before applying machine learning models. EDA typically includes various plotting techniques such as histograms, box plots, and scatter plots.



OBJECTIVE

The goal of this Project is to perform EDA on a car dataset from Kaggle, which includes over 10,000 rows and more than 10 columns of features such as Engine Fuel Type, Engine HP, Transmission Type, and MPG values. Through EDA, we will explore this data and prepare it for further modeling.

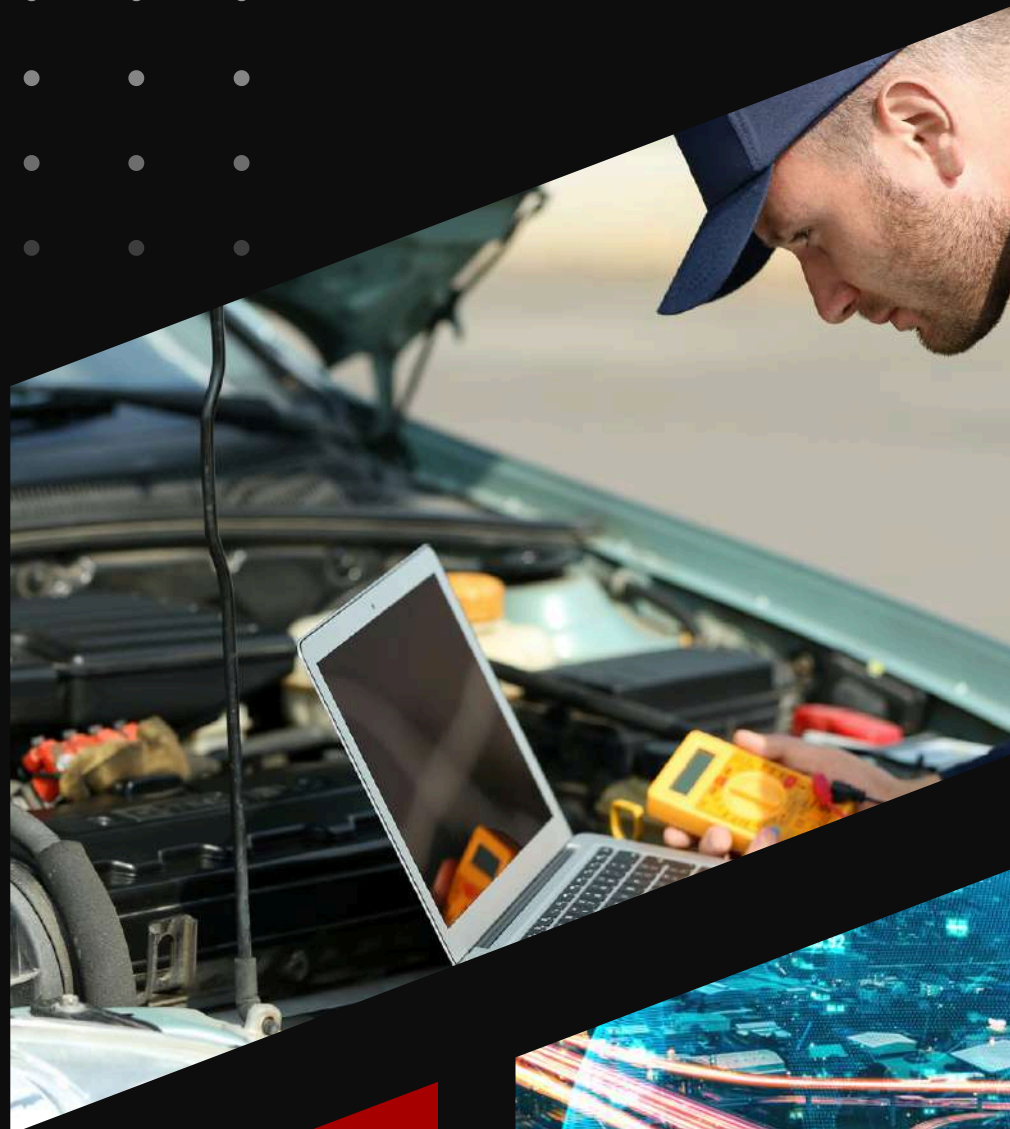
CAR DATA SET

+10,000

Rows

+10

Columns





DATA OVERVIEW

About data

To give a piece of brief information about the data set this data contains more of 10, 000 rows and more than 10 columns which contains features of the car such as Engine Fuel Type, Engine HP, Transmission Type, highway MPG, city MPG and many more. So in this tutorial, we will explore the data and make it ready for modeling.

Data set source

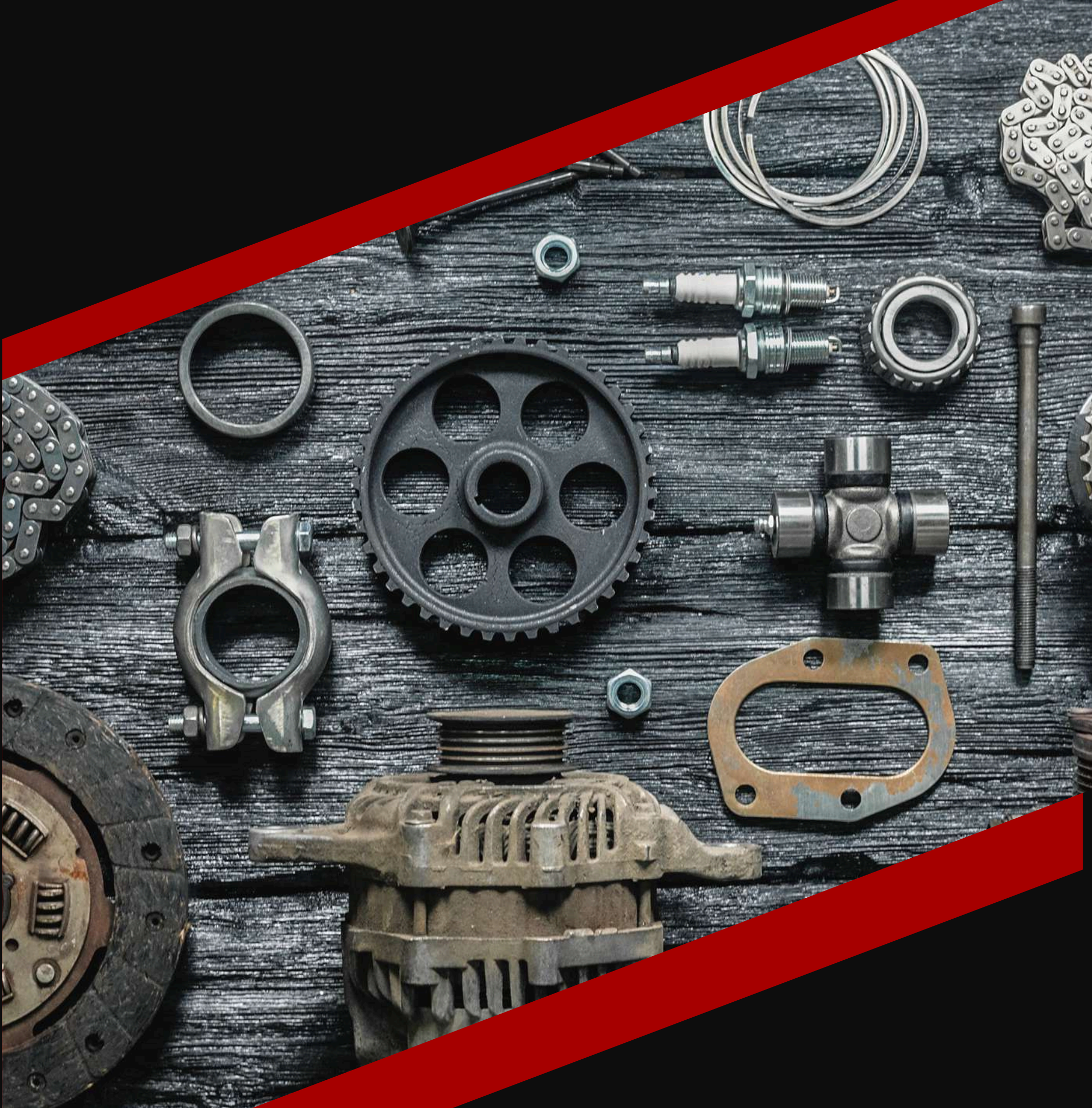
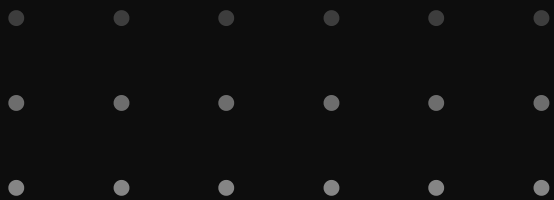
The dataset used in this analysis is the 'Car Features and MSRP' dataset, which was obtained from Kaggle.





COLUMNS

- Make: object
- Model: object
- Year: int64
- Engine Fuel Type: object
- Engine HP: float64
- Engine Cylinders: float64
- Transmission Type: object
- Driven_Wheels: object
- Number of Doors: float64
- Market Category: object
- Vehicle Size: object
- Vehicle Style: object
- highway MPG: int64
- city mpg: int64
- Popularity: int64
- MSRP: int64



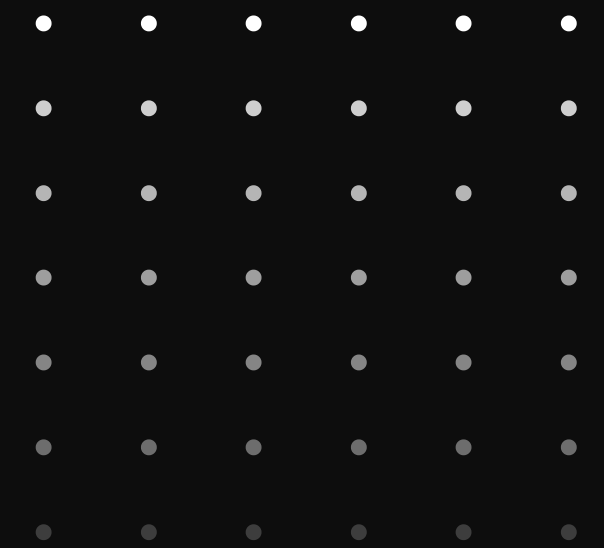


DATA LOADING AND CLEANING

Data Loading:

- The dataset was successfully loaded into a Pandas DataFrame using the `pd.read_csv()` function.
- The initial shape of the dataset was determined, indicating the number of rows and columns.

```
# 2. Loading the data into the data frame.  
df = pd.read_csv("data.csv")  
  
#to find the shape of the data  
df.shape  
✓ 0.0s
```



DATA LOADING AND CLEANING

Dropping Irrelevant Columns:

- Several columns that were deemed irrelevant for the analysis were dropped from the dataset. These columns included:
 - Engine Fuel Type, Market Category, Vehicle Style, Popularity, Number of Doors, Vehicle Size
- After dropping these columns, the dataset became more focused on the essential features needed for further analysis.

```
#Dropping irrelevant columns
df = df.drop(['Engine Fuel Type', 'Market Category', 'Vehicle Style', 'Popularity', 'Number of Doors', 'Vehicle Size'], axis=1)
df.head(5)
```

✓ 0.0s



DATA LOADING AND CLEANING

Handling Missing Values:

- The dataset was checked for missing or null values using the `isnull().sum()` function.
- Any columns or rows containing missing values were dropped, ensuring that the dataset is clean and ready for analysis.
- After the cleaning process, the dataset was free of missing values.

```
# Dropping the missing or null values.  
print(df.isnull().sum())
```

✓ 0.0s

```
df = df.drop_duplicates()  
df.head(5)
```

✓ 0.0s



DESCRIPTIVE STATISTICS

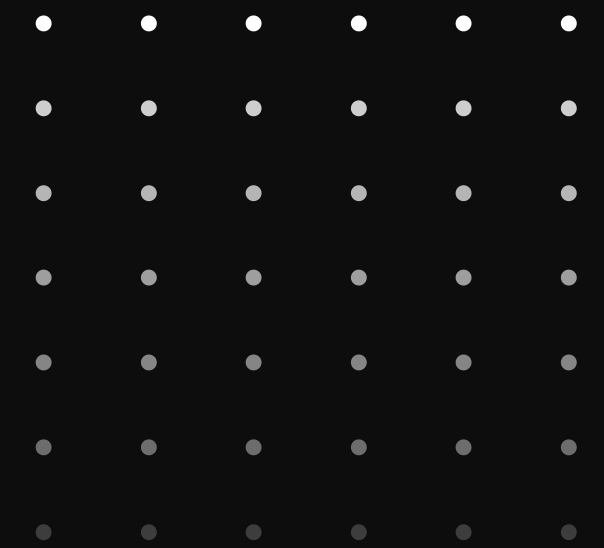
- can use the describe() function. This function provides a summary of the central tendency, dispersion, and shape of a dataset's distribution, excluding NaN values.

```
descriptive_stats = df.describe()
descriptive_stats
```

✓ 0.0s

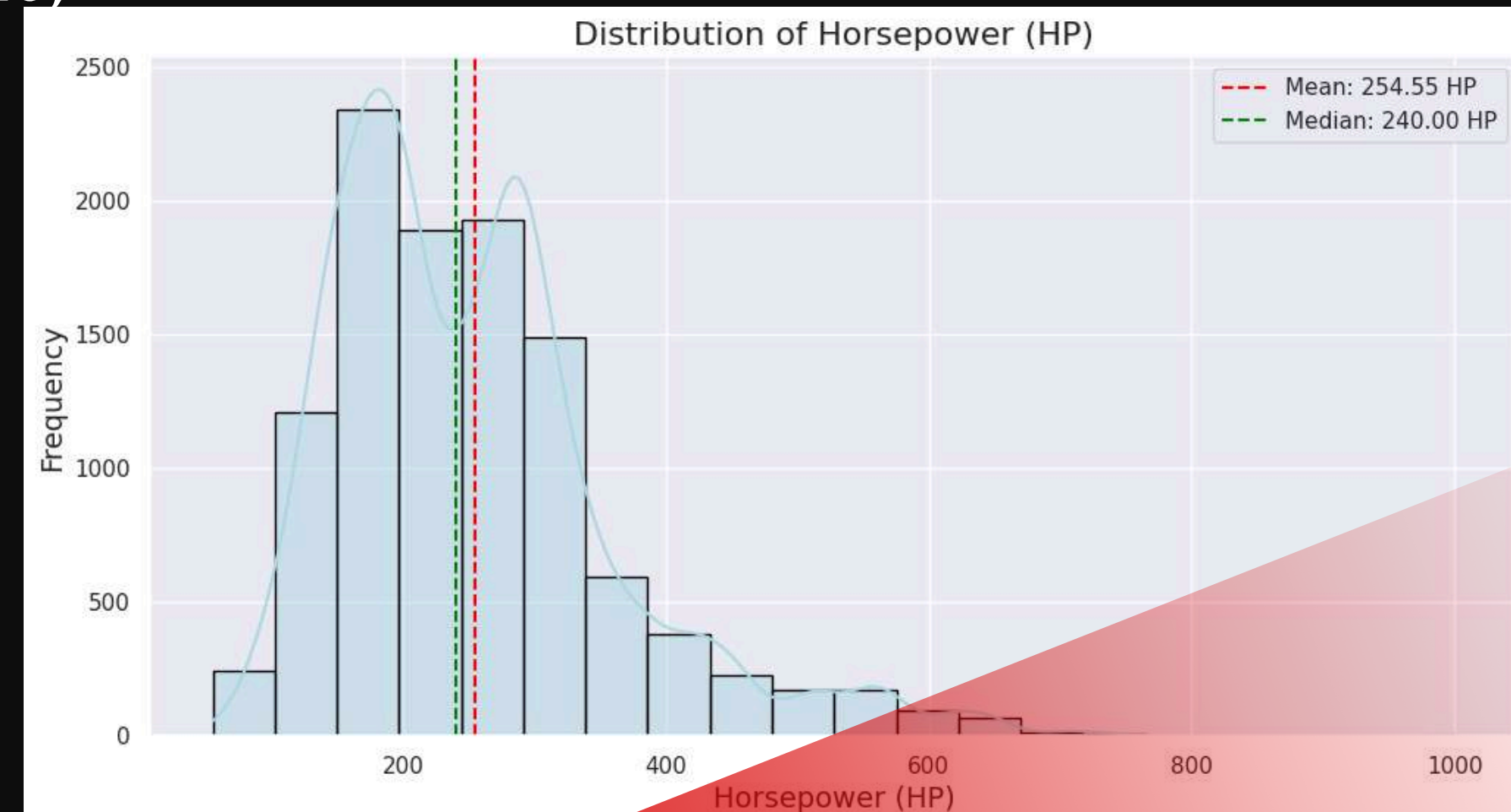
	Year	HP	Cylinders	MPG-H	MPG-C	Price
count	9191.000000	9191.000000	9191.000000	9191.000000	9191.000000	9191.000000
mean	2011.328909	236.735285	5.436514	26.432053	19.251877	30976.473615
std	6.276064	76.816966	1.388069	5.715501	4.366754	14992.073695
min	1994.000000	63.000000	3.000000	13.000000	10.000000	2000.000000
25%	2008.000000	175.000000	4.000000	22.000000	16.000000	22095.000000
50%	2015.000000	230.000000	6.000000	26.000000	18.000000	29990.000000
75%	2016.000000	290.000000	6.000000	30.000000	22.000000	39730.000000
max	2017.000000	485.000000	8.000000	42.000000	31.000000	75200.000000

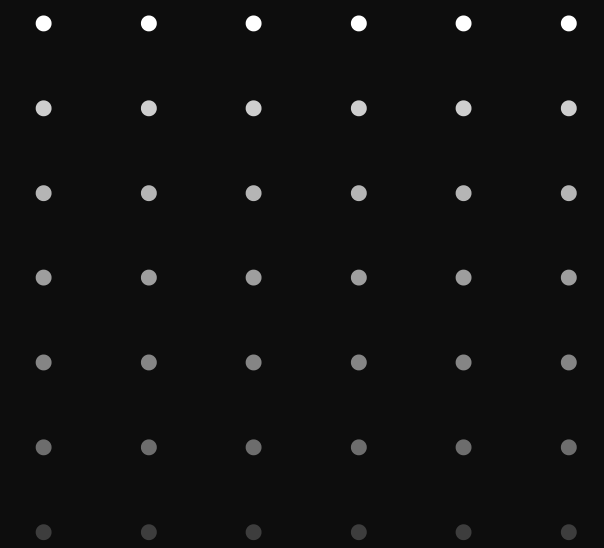




DATA ANALYSIS & VISUALIZATION

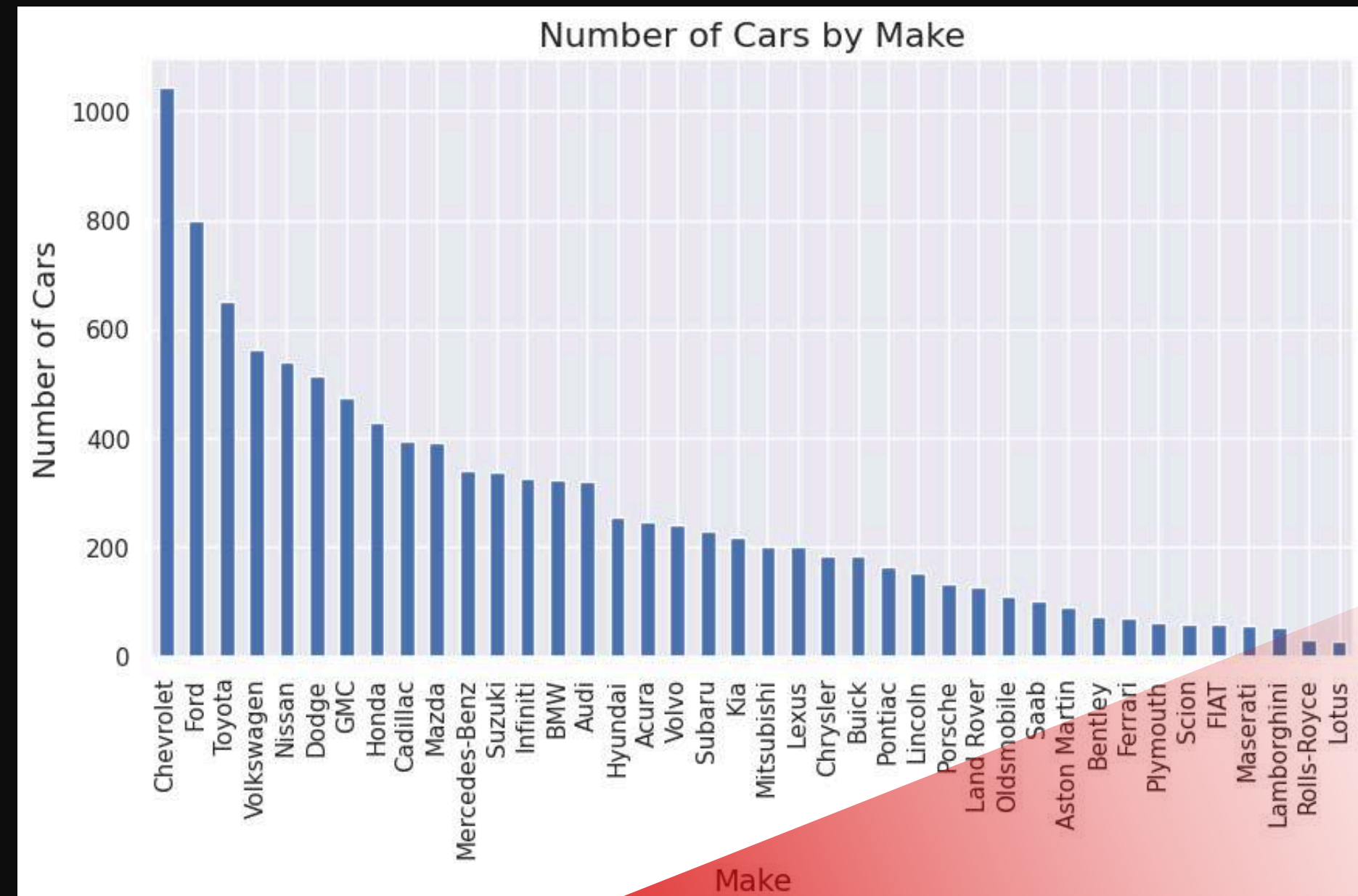
- `# Plot distribution of Horsepower (HP)`
- `plt.figure(figsize=(12, 6))`
- `sns.histplot(df['HP'], bins=20, kde=True, color='lightblue', edgecolor='black')`
- `plt.axvline(df['HP'].mean(), color='red', linestyle='--', label=f'Mean: {df["HP"].mean():.2f} HP')`
- `plt.axvline(df['HP'].median(), color='green', linestyle='--', label=f'Median: {df["HP"].median():.2f} HP')`
- `plt.title('Distribution of Horsepower (HP)', fontsize=16)`
- `plt.xlabel('Horsepower (HP)', fontsize=14)`
- `plt.ylabel('Frequency', fontsize=14)`
- `plt.legend()`
- `plt.show()`





DATA ANALYSIS & VISUALIZATION

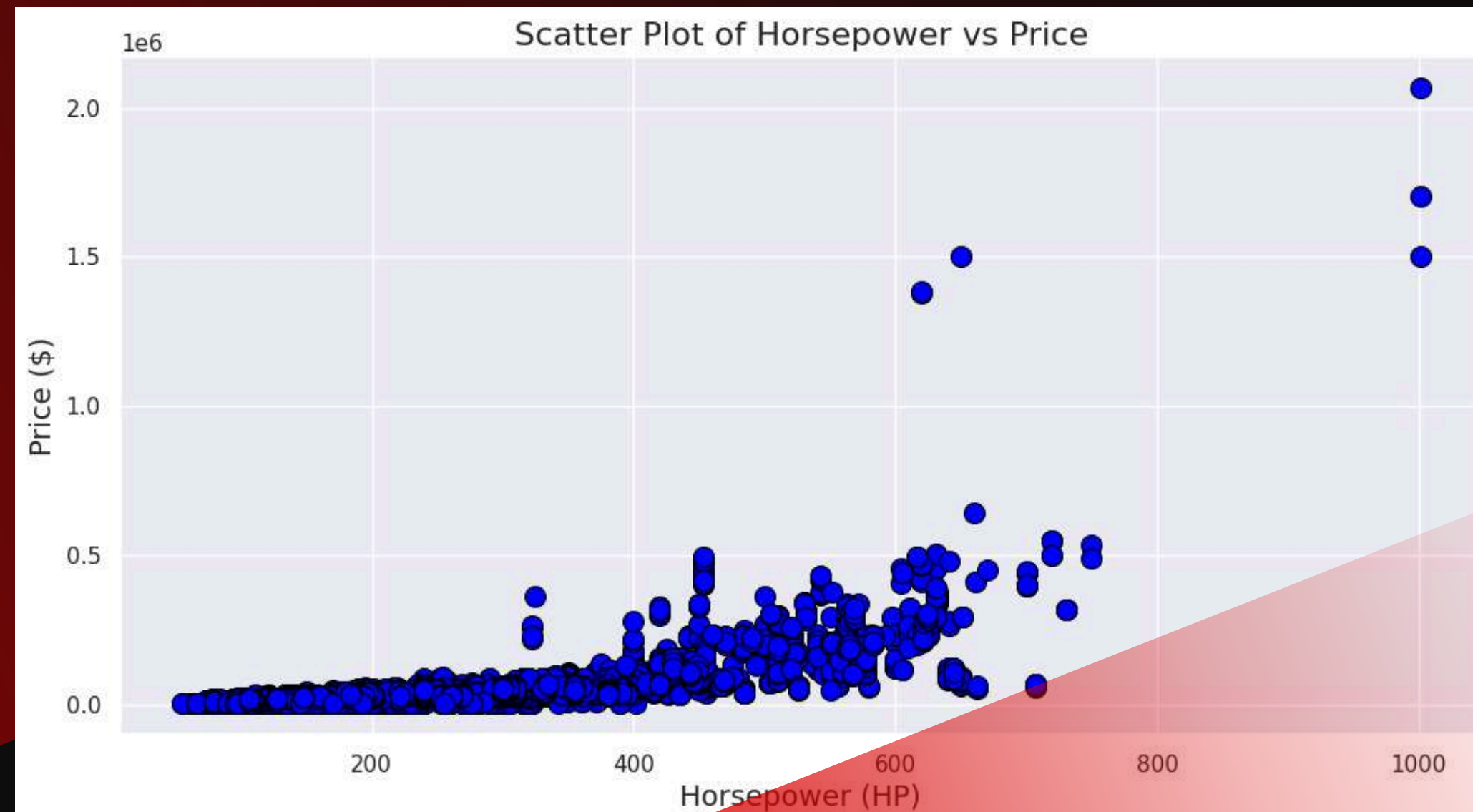
- # Bar plot for the number of cars by Make
- `plt.figure(figsize=(10, 5))`
- `df.Make.value_counts().nlargest(40).plot(kind='bar')`
- `plt.title("Number of Cars by Make", fontsize=16)`
- `plt.ylabel('Number of Cars', fontsize=14)`
- `plt.xlabel('Make', fontsize=14)`
- `plt.show()`
-





BIVARIATE ANALYSIS

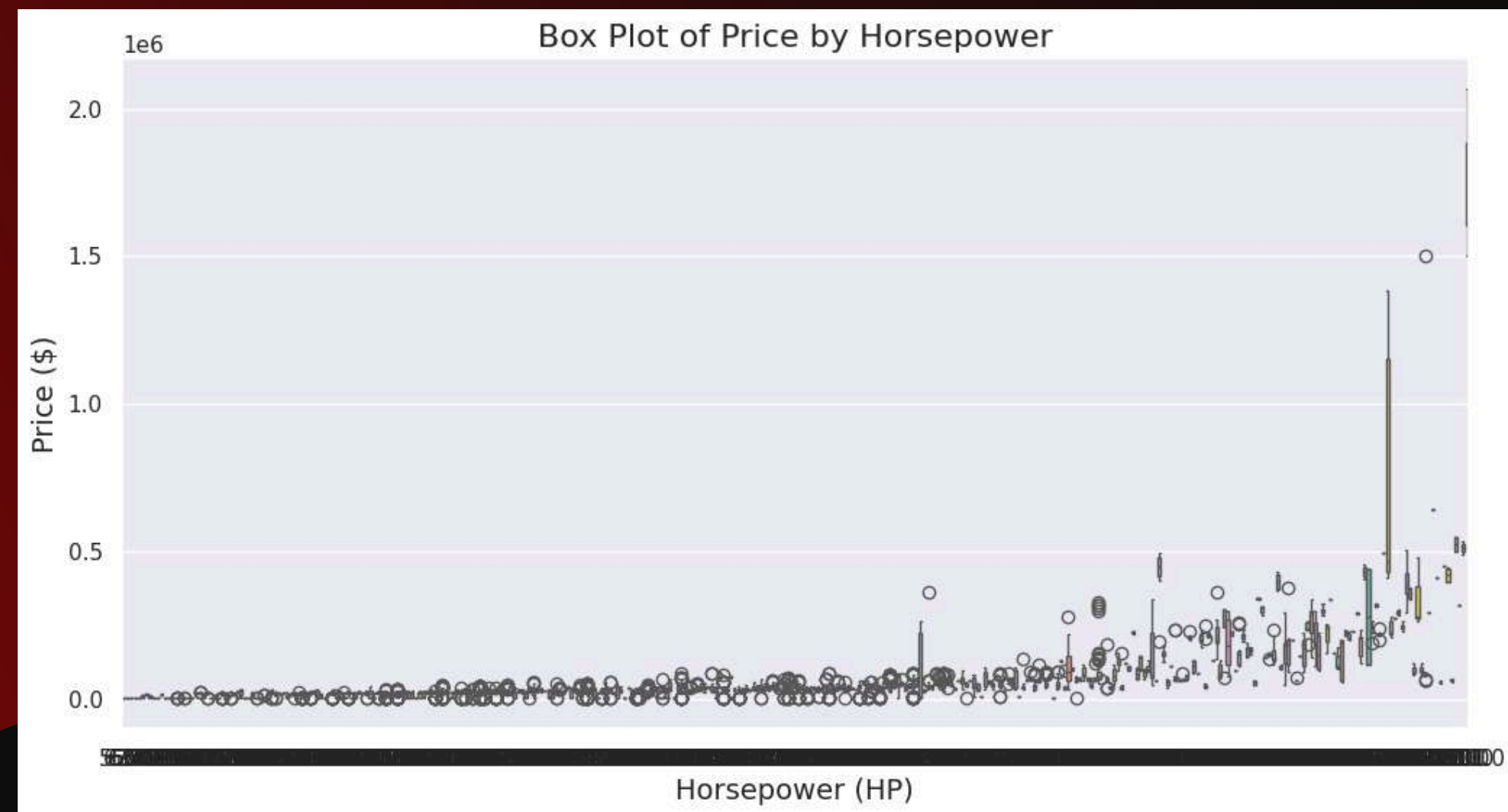
- # Scatter plot of Horsepower vs Price
- `plt.figure(figsize=(12, 6))`
- `sns.scatterplot(x='HP', y='Price', data=df, color='blue', s=100, edgecolor='black')`
- `plt.title('Scatter Plot of Horsepower vs Price', fontsize=16)`
- `plt.xlabel('Horsepower (HP)', fontsize=14)`
- `plt.ylabel('Price ($)', fontsize=14)`
- `plt.grid(True)`
- `plt.show()`



BIVARIATE ANALYSIS



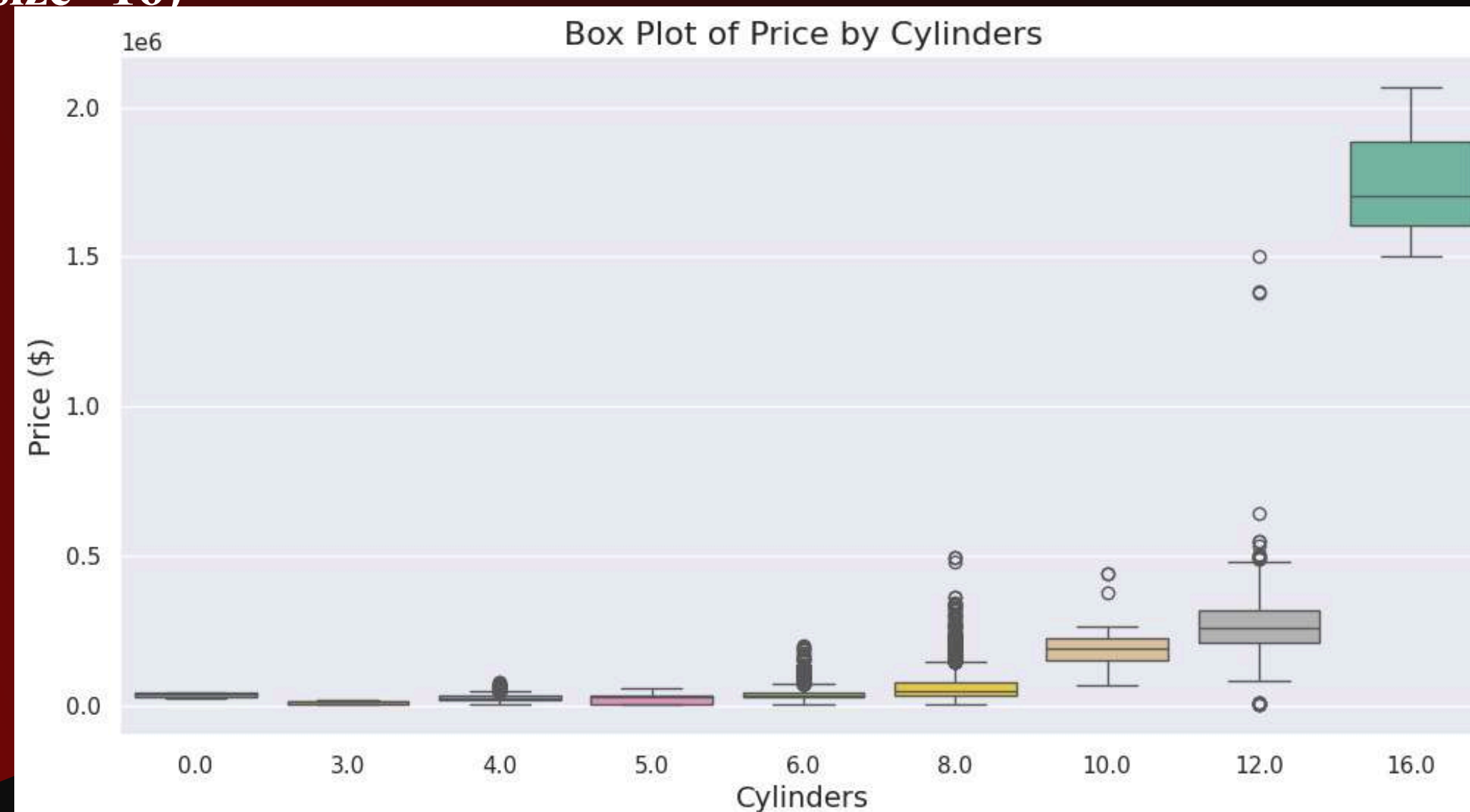
- # Box plot of Price by Horsepower
- `plt.figure(figsize=(12, 6))`
- `sns.boxplot(x='HP', y='Price', data=df, palette='Set2')`
- `plt.title('Box Plot of Price by Horsepower', fontsize=16)`
- `plt.xlabel('Horsepower (HP)', fontsize=14)`
- `plt.ylabel('Price ($)', fontsize=14)`
- `plt.show()`



BIVARIATE ANALYSIS



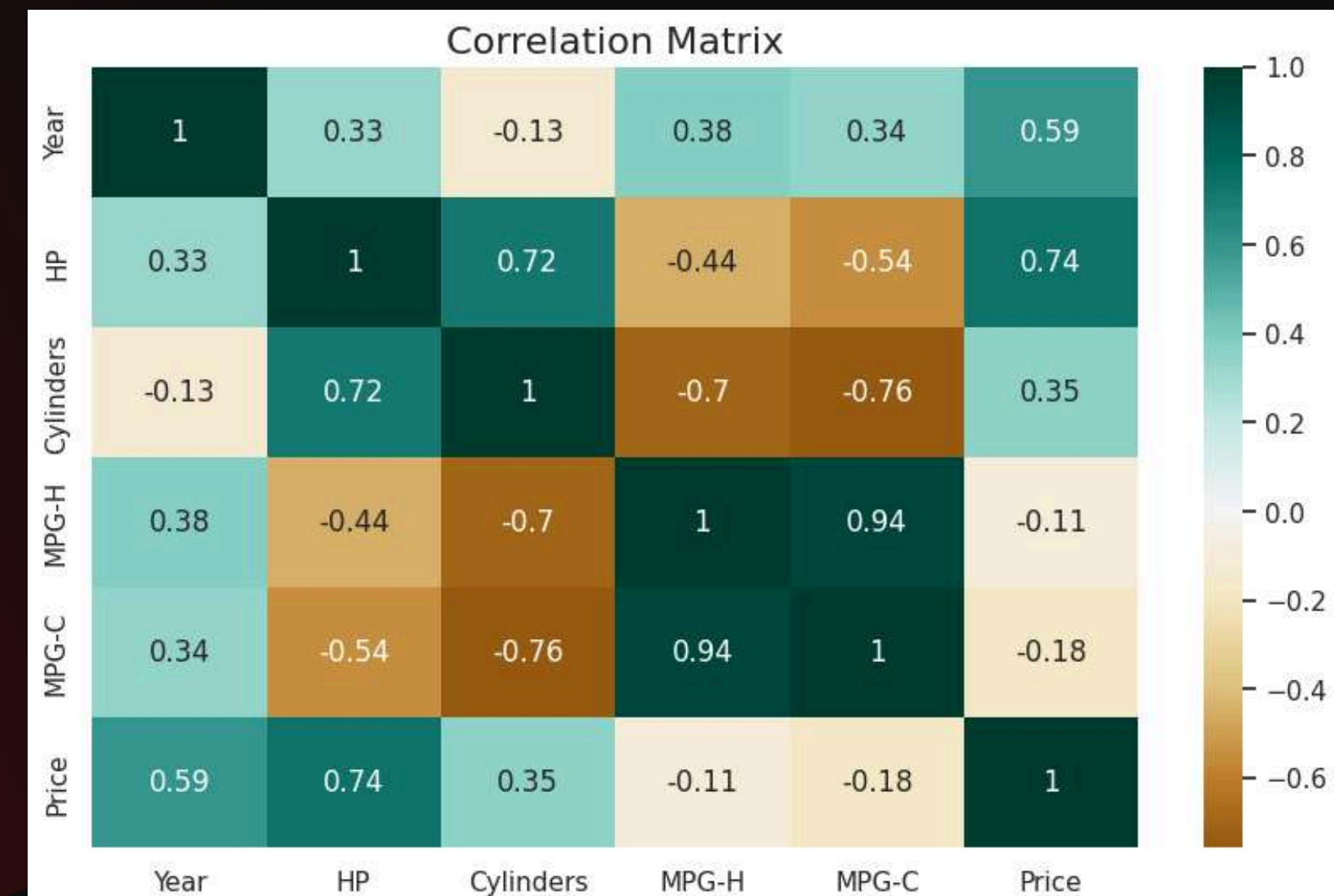
- # Box plot of Price by Cylinders
- `plt.figure(figsize=(12, 6))`
- `sns.boxplot(x='Cylinders', y='Price', data=df, palette='Set2')`
- `plt.title('Box Plot of Price by Cylinders', fontsize=16)`
- `plt.xlabel('Cylinders', fontsize=14)`
- `plt.ylabel('Price ($)', fontsize=14)`
- `plt.show()`
-

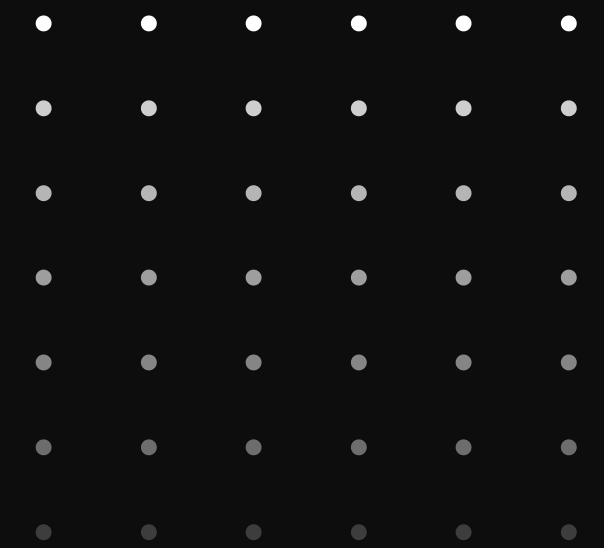




CORRELATION ANALYSIS (MULTIVARIATE ANALYSIS)

- # Correlation matrix
- `numeric_df = df.select_dtypes(include=['float64', 'int64'])`
- `correlation_matrix = numeric_df.corr()`
- `plt.figure(figsize=(10, 6))`
- `sns.heatmap(correlation_matrix, annot=True, cmap='BrBG', center=0)`
- `plt.title('Correlation Matrix', fontsize=16)`
- `plt.show()`





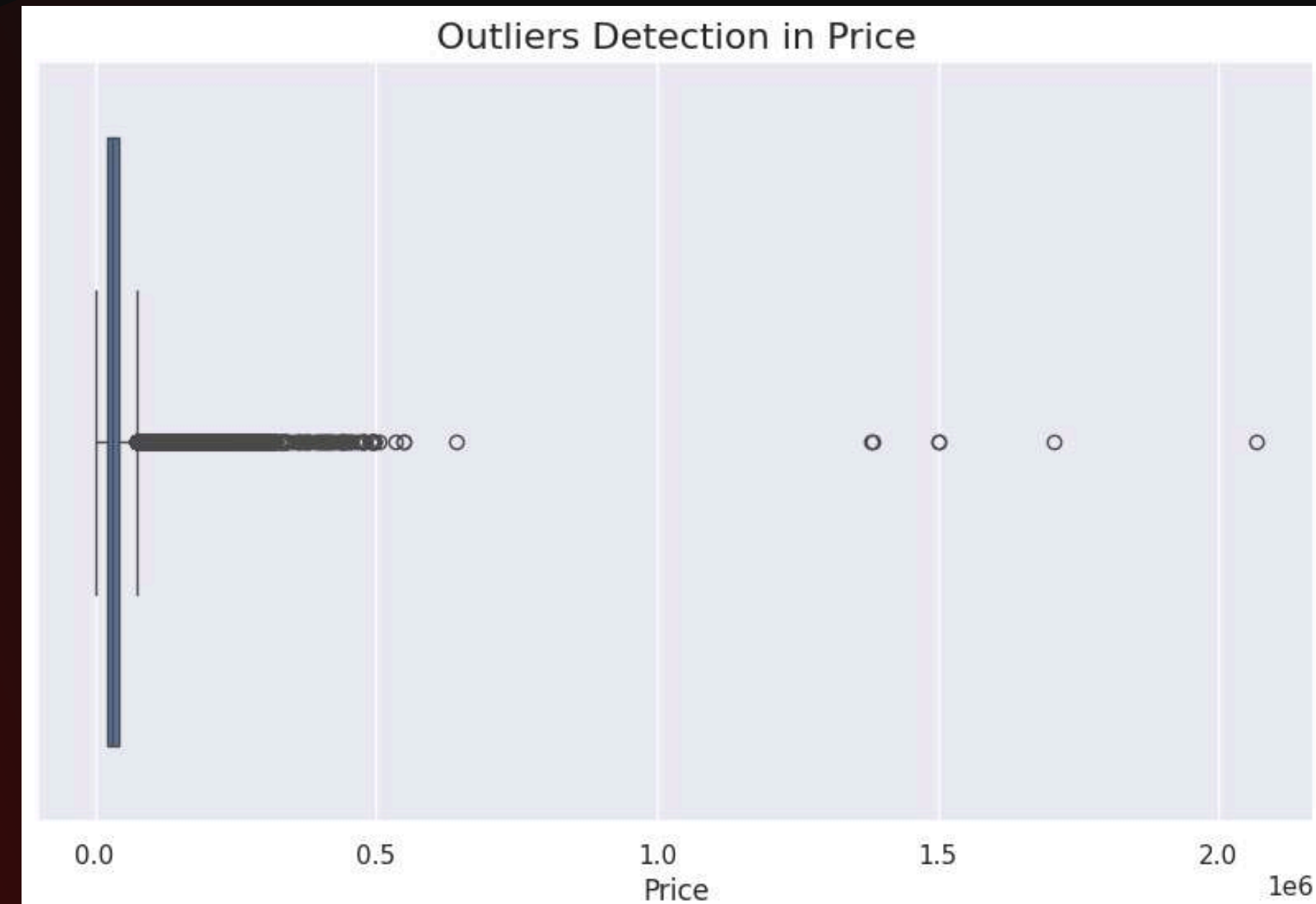
ADVANCED BINNING AND GROUPING

- # Categorize Horsepower (HP) into bins
- `df['HP_Category'] = pd.cut(df['HP'], bins=3, labels=['Low', 'Medium', 'High'])`
- # Bar plot of Mean Price by HP Categories
- `plt.figure(figsize=(12, 6))`
- `sns.barplot(x='HP_Category', y='Price', data=df, palette='viridis')`
- `plt.title('Mean Price by Horsepower Categories', fontsize=16)`
- `plt.xlabel('Horsepower Category', fontsize=14)`
- `plt.ylabel('Mean Price ($)', fontsize=14)`
- `plt.show()`



OUTLIER DETECTION AND HANDLING

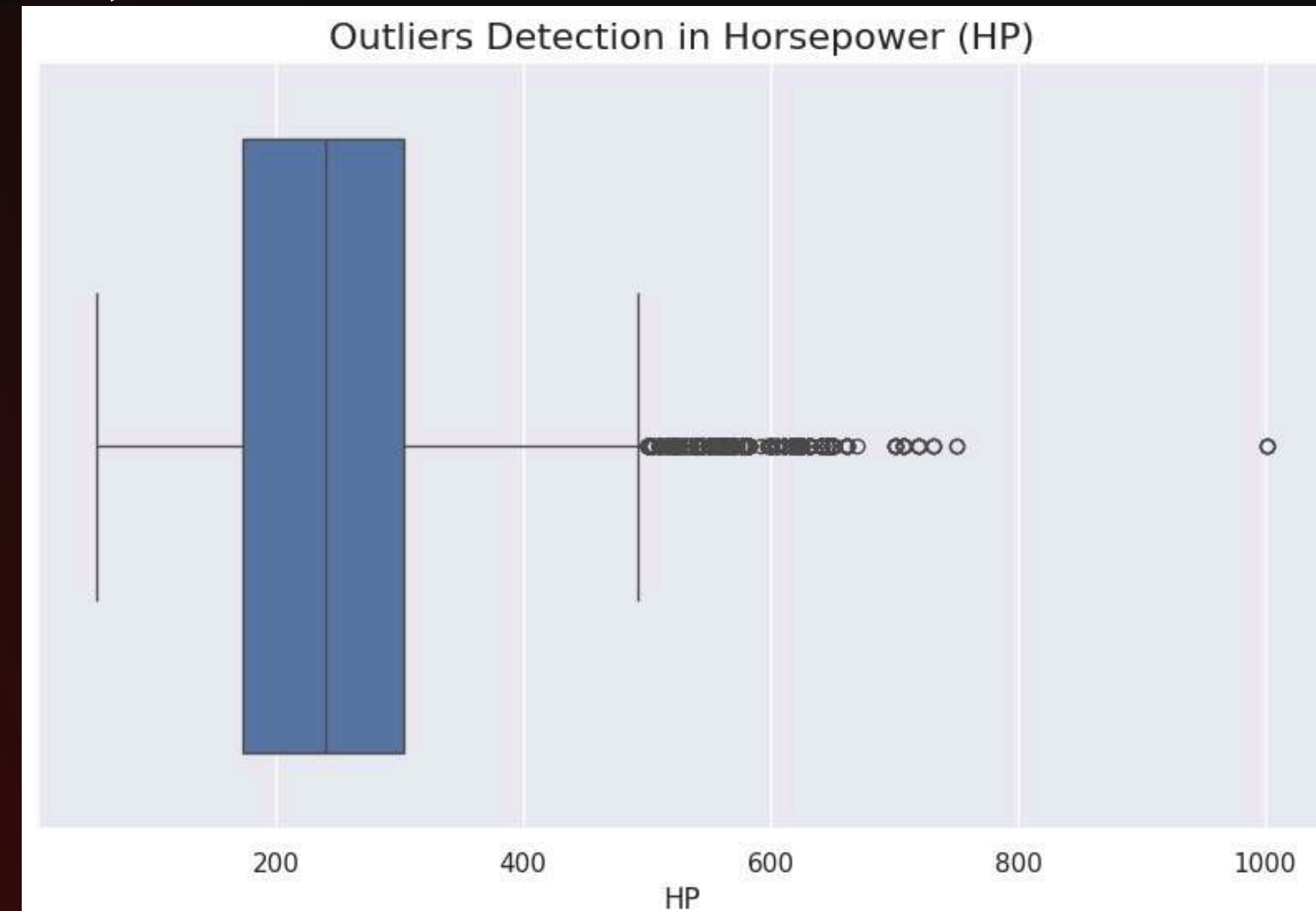
- # Detect outliers in Price using Boxplot
- `plt.figure(figsize=(10, 6))`
- `sns.boxplot(x=df['Price'])`
- `plt.title('Outliers Detection in Price', fontsize=16)`
- `plt.show()`

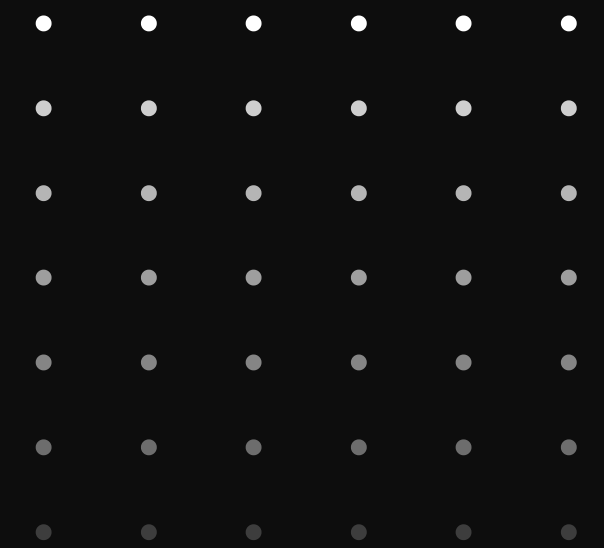




OUTLIER DETECTION AND HANDLING

- `# Detect outliers in Horsepower (HP)`
- `plt.figure(figsize=(10, 6))`
- `sns.boxplot(x=df['HP'])`
- `plt.title('Outliers Detection in Horsepower (HP)', fontsize=16)`
- `plt.show()`





OUTLIER DETECTION AND HANDLING

- # Outlier removal using IQR
- `Q1 = numeric_df.quantile(0.25)`
- `Q3 = numeric_df.quantile(0.75)`
- `IQR = Q3 - Q1`
- `df_clean = df[~((numeric_df < (Q1 - 1.5 * IQR)) | (numeric_df > (Q3 + 1.5 * IQR))).any(axis=1)]`
- `df_clean.shape`



INSIGHTS

- **Horsepower and Price:** Strong positive correlation—higher horsepower generally leads to higher prices.
- **Price and Cylinders:** Cars with more cylinders tend to be priced higher.
- **Brand Dominance:** Ford, Chevrolet, and Toyota are the most represented brands, potentially influencing trends.
- **Outliers:** Significant outliers in both horsepower and price suggest the presence of luxury and performance vehicles.
- **Fuel Efficiency:** Higher horsepower cars tend to have lower fuel efficiency



KEY RECOMMENDATIONS

- **Outlier Treatment:** Address outliers in price and horsepower for accurate modeling and predictions.
- **Brand Bias:** Consider the heavy presence of a few brands when analyzing trends—balance the dataset if possible.
- **Market Segmentation:** Segment vehicles based on horsepower categories to better understand price dynamics.
- **Efficiency-Performance Trade-Off:** Account for the trade-off between performance (horsepower) and fuel efficiency in pricing strategies.



SANKAR M



vmgomathisankar@gmail.com



Rajapalayam, TamilNadu 626117





THANK YOU

